Process Management

- Operating Systems
- Detailed Explanation with Real-Life Examples

What is a Process?

- A process is a program in execution.
- Program: Passive (stored on disk)
- Process: Active (running in memory)

- Example:
- Recipe = Program
- Cooking = Process

Process States

- New
- Ready
- Running
- Waiting (Blocked)
- Terminated

Process State – New & Ready

- New: Process is being created
- Ready: Process waiting for CPU

- Example:
- Opening MS Word
- Waiting in a bank queue

Process State – Running & Waiting

- Running: CPU executing process
- Waiting: Waiting for I/O

- Example:
- Talking to bank clerk
- Filling form

Process State – Terminated

- Process finished execution

- Example:
- Leaving bank after work done

Process State Transitions

- New → Ready → Running → Terminated
- Running → Waiting → Ready

- Example:
- Traffic signal system

Process Control Block (PCB)

- PCB stores all process information

- Contents:
- PID
- State
- Program Counter
- CPU Registers
- Memory Info
- I/O Info

PCB Real-Life Example

- Hospital patient file:
- Patient ID → PID
- Condition → State
- Reports → Registers

Context Switching

- Saving state of one process
- Loading state of another process

- Used in multitasking systems

Context Switching Example

- Teacher switching between classes
- Notes saved and resumed later

- Disadvantage: CPU overhead

Threads

- Smallest unit of execution
- Multiple threads in one process

- Example:
- Restaurant → Process
- Waiters → Threads

Multithreading

- Multiple threads running concurrently

- Example:
- YouTube app:
- Video playback
- Buffering
- Comments

Thread Models

- User-Level Threads
- Kernel-Level Threads
- Multilevel Threads

Thread Mapping Models

- One-to-One
- Many-to-One
- Many-to-Many

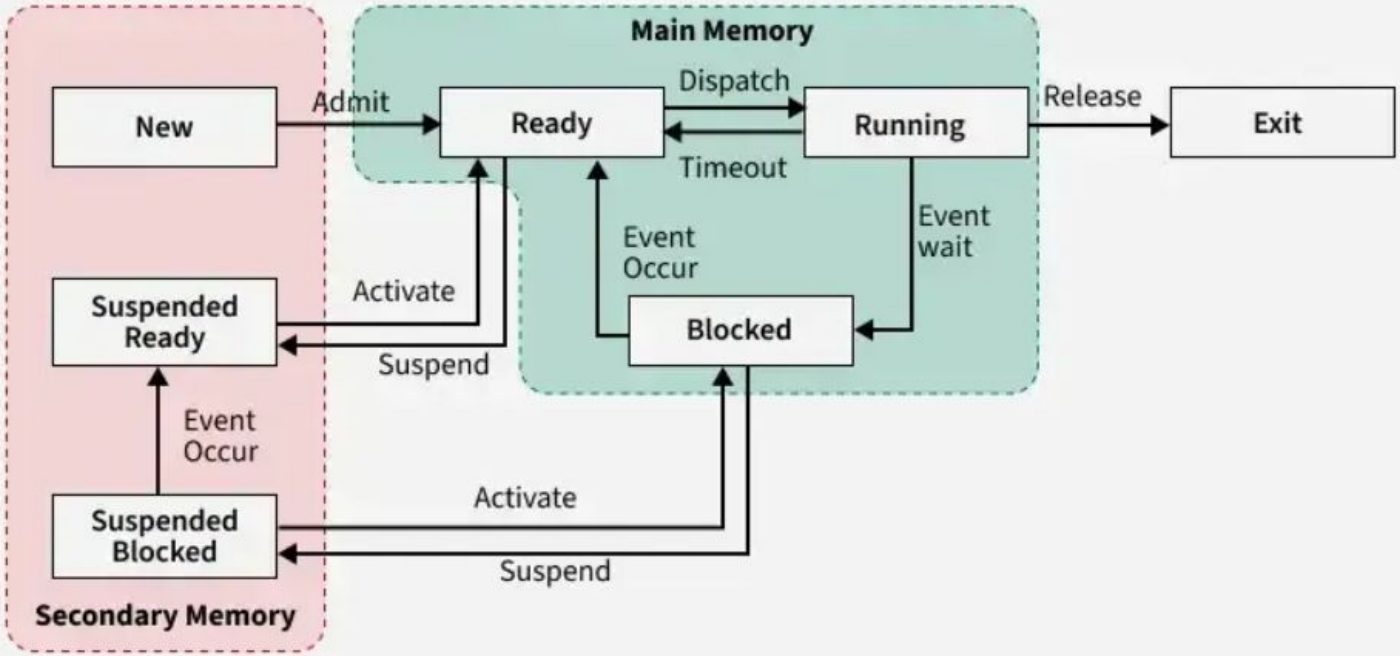
Benefits of Threads

- Responsiveness
- Resource Sharing
- Economy
- Scalability

Process scheduling

- Process scheduling is the activity of the process manager that handles the removal of the running process from the CPU and the selection of another process based on a particular strategy. Throughout its lifetime, a process moves between various scheduling queues, such as the ready queue, waiting queue or devices queue.

- Scheduling is important in operating systems with multiprogramming as multiple processes might be eligible for running at a time.
- One of the key responsibilities of an Operating System (OS) is to decide which programs will execute on the CPU.
- Process Schedulers are fundamental components of operating systems responsible for deciding the order in which processes are executed by the CPU. In simpler terms, they manage how the CPU allocates its time among multiple tasks or processes that are competing for its attention.
-

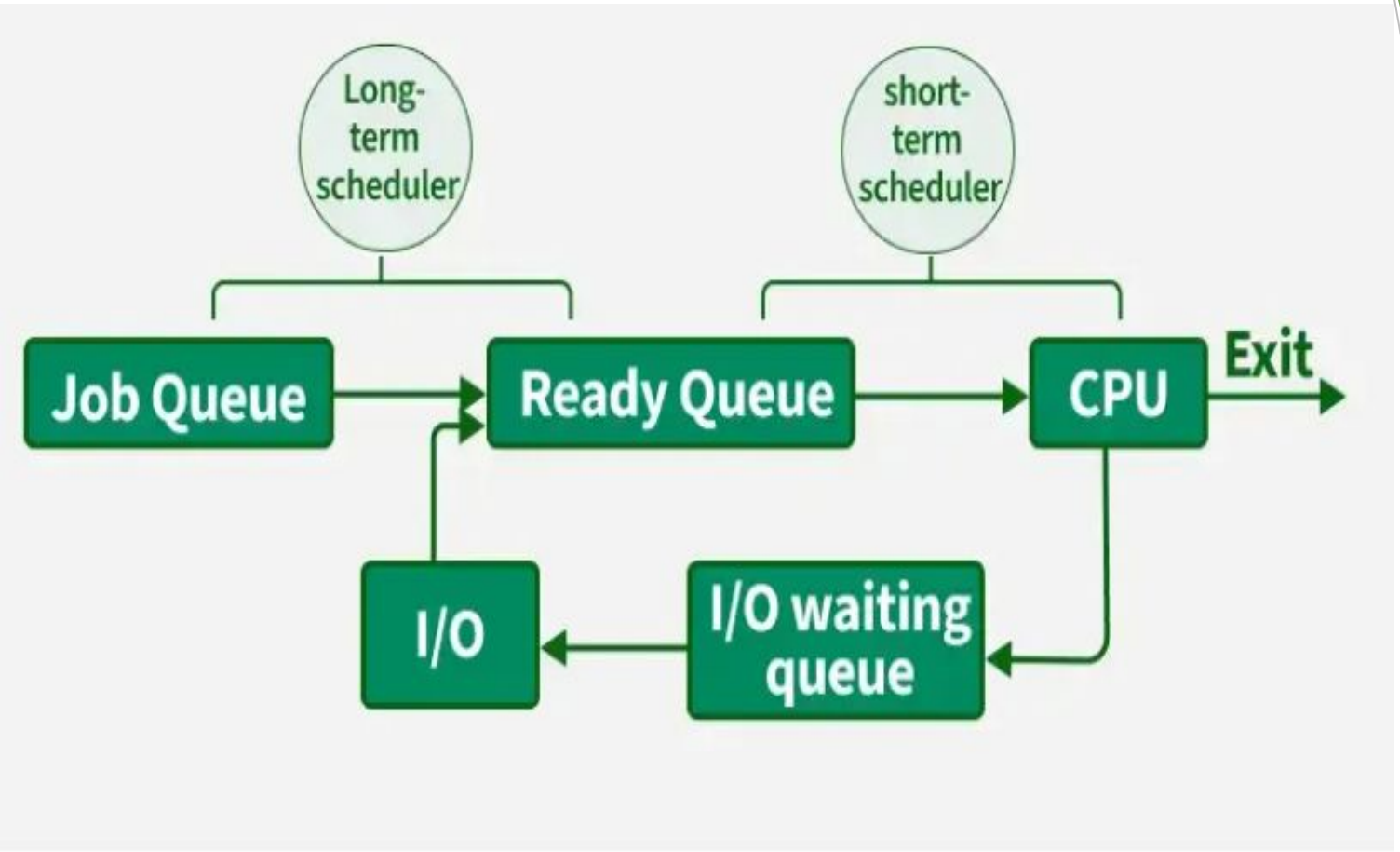

Main Memory
New
Admit
Ready
Dispatch
Running
Release
Exit
Timeout
Event
Occur
Event
wait
Suspended
Ready
Activate
Suspend
Blocked
Event
Occur
Activate
Suspended
Blocked
Suspend
Secondary Memory

LTS

- The Long-Term Scheduler is responsible for loading processes from disk into main memory so they can begin execution.
- Transfers processes from the Job Queue to the Ready Queue.
- Controls the degree of Multi-programming — the number of processes present in memory or ready state at any time.
- Carefully selects a balanced mix of I/O-bound and CPU-bound processes to ensure efficient system performance.
- Helps avoid a situation where either the CPU or I/O devices remain idle.
- In many modern time-sharing systems (such as Windows), a long-term scheduler may not exist; new processes are directly admitted to memory for short-term scheduling.
- It is the **slowest** among all schedulers, as it operates less frequently.
-

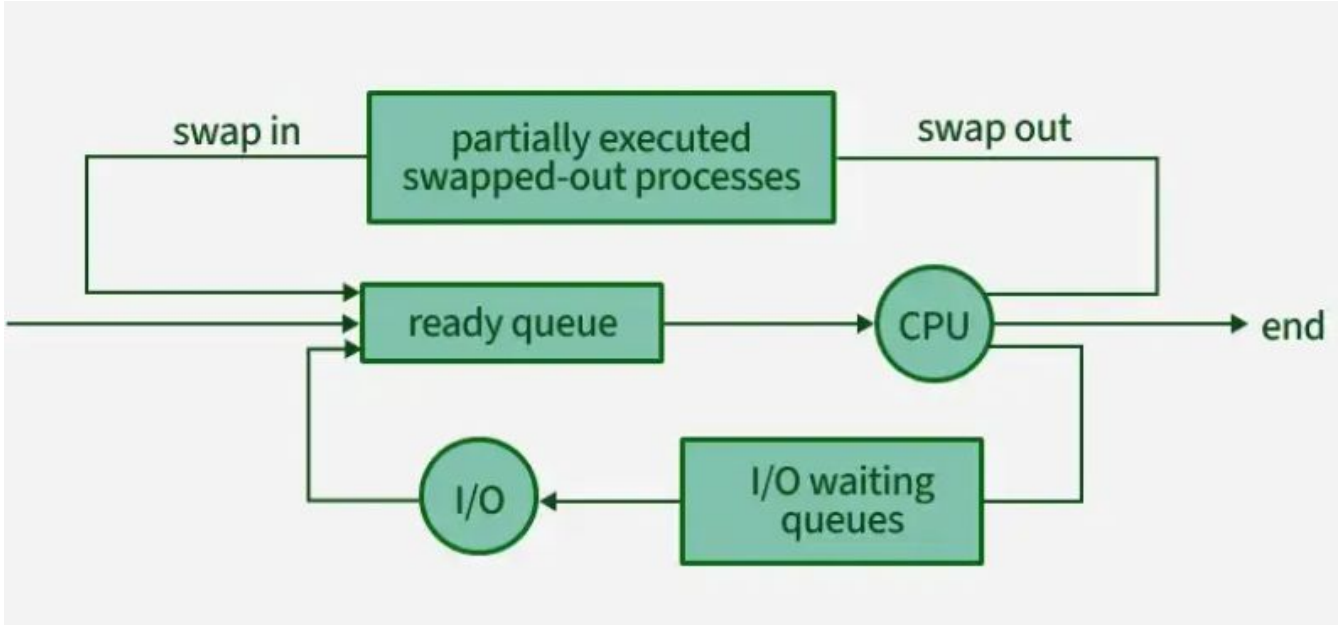
Short-Term Scheduler

- The Short-Term Scheduler (STS) is responsible for selecting a process from the ready queue and assigning the CPU to it.
- Frequently selects the next process to execute from the ready state.
- Ensures no process suffers from starvation.
- Uses various CPU scheduling algorithms to decide process order.
- Maximizes CPU utilization by keeping the processor as busy as possible.
- Calls the dispatcher, which performs the actual context switch.
- It is the **fastest** scheduler, since it operates very frequently


Long-
term
scheduler
short-
term
scheduler
Job Queue
Ready Queue
CPU
Exit
I/O
I/O waiting
queue

MTS

- The Medium-Term Scheduler (MTS) manages swapping, which temporarily moves processes between main memory and disk.
- Swaps processes out of memory when they are waiting (e.g., blocked for I/O) to reduce the degree of multiprogramming.
- Frees memory for other active processes.
- Swaps processes back into memory when they are ready to continue execution, allowing them to resume from where they left off.
- Helps maintain an effective mix of CPU-bound and I/O-bound processes.
- It operates faster than the long-term scheduler but slower than the short-term scheduler.


swap in
partially executed swapped-out processes
swap out
ready queue
CPU
end
I/O
I/O waiting queues

Long-Term Scheduler	Short-Term Scheduler	Medium-Term Scheduler
It is a job scheduler	It is a CPU scheduler	It is a process-swapping scheduler
It is the slowest scheduler	It is the fastest scheduler	Its speed lies between long-term and short-term schedulers
It controls the degree of multiprogramming	It gives less control over the degree of multiprogramming	It reduces the degree of multiprogramming
It is barely present or nonexistent in time-sharing systems	It is essential for time-sharing systems	It is a component of time-sharing systems
It selects processes from the job pool and loads them into memory	It selects ready processes for execution by the CPU	It can re-introduce processes into memory and resume execution